

FPGA-Based Hardware Accelerator for LWE Encryption and Decryption with TFHE Scheme

Najmi Azzahra Feryputri[‡], Infall Syafalni^{‡§¶}, Nana Sutisna^{‡§}, Nur Ahmadi^{‡§},
Rahmat Mulyawan^{‡§}, and Trio Adiono^{‡§}

[‡]School of Electrical Engineering and Informatics, Bandung Institute of Technology, Indonesia

[§]University Center of Excellence on Microelectronics, Bandung Institute of Technology, Indonesia

[¶]Systems Design Lab, School of Engineering, The University of Tokyo, Tokyo, Japan

Abstract—Fully Homomorphic Encryption (FHE) allows computations on encrypted data, ensuring data confidentiality. This paper presents an FPGA-based accelerator for Learning With Error (LWE) encryption and decryption in the Torus FHE (TFHE) scheme. The design processes plaintext in parallel and supports larger sizes sequentially, where the number of bits in plaintext is 8 bits to 64 bits. The proposed method maps the input to the torus domain by several modules such as *fpToTorus32*, *modToTorus32*, *bitTorus32*, and *accumulator* as well as the pseudo-random number generator (PRNG). Implemented on a PYNQ-Z1 FPGA, the proposed design outperforms existing reference software in speed while maintaining low resource utilization and power consumption. Experimental results show that the proposed encryption achieves up to $42\times$ speed up and the proposed decryption achieves up to $10\times$ speed up compared to the reference software running in an AMD Ryzen 7 processor with 16GB memory. The results highlight the potential of FPGA acceleration for efficient and practical homomorphic encryption.

Index Terms—FHE, TFHE, LWE, FPGA, Hardware Accelerator, Encryption, Decryption.

I. INTRODUCTION

Encryption is one of the commonly used techniques to maintain data confidentiality. In 1978, Rivest and his colleagues proposed an encryption concept that allowed operations to be performed on encrypted data. Rivest called this encryption scheme Privacy Homomorphisms [1]. In the same year, Rivest, along with Adi Shamir and Leonard Adleman, developed the RSA cryptosystem [2]. This scheme was the first to demonstrate how to multiply two ciphertexts.

In 2009, Gentry proposed the first Fully Homomorphic Encryption scheme as a topic of his dissertation [3]. This scheme allowed both multiplication and addition to be performed homomorphically. This discovery was quite phenomenal and opened new doors for research in Fully Homomorphic Encryption. From 2010 to the present, research on Fully Homomorphic Encryption has continued. In 2015, Ducas and Micciancio designed a Boolean-based FHE scheme that operates by evaluating Boolean gates on encrypted bits [4]. Shortly thereafter, in 2016, Ilaria Chillotti, Nicolas Gama, Mariya Georgieva, and Malika Izabachène improved Ducas-Micciancio's bootstrapping scheme. The bootstrapping process was considered a collection of 'external products' [5]. The scheme developed by Ilaria

Chillotti is known as Fully Homomorphic Encryption Over the Torus or TFHE.

TFHE consists of three types of ciphertexts: LWE (Learning with Errors), TLWE (Torus Learning with Errors), and TGSW (Torus Generalized Smart-Vercauteren). LWE is the most fundamental ciphertext, used to encrypt a single unique character or a single bit of a message. TLWE is used to encrypt a group of bits from a message. TGSW ciphertext is used for bootstrapping purposes.

The latest research on the hardware implementation of TFHE primarily focuses on the bootstrapping process. Research in [6] implements an architecture capable of performing parallel computations for bootstrapping using the Virtex UltraScale+ VCU128 FPGA platform. The study in [7] implements bootstrapping by designing an FPGA accelerator that exhibits high area efficiency and low latency. Research in [8] implements bootstrapping at the register transfer level (RTL) on the ZYNQ ZCU102 FPGA board. The only research related to encryption and decryption schemes in TFHE is the C-based software developed by Ilaria Chillotti along with Zama researchers in [9]. Moreover, to the best of our knowledge, no research has implemented TFHE encryption and decryption in the FPGAs or ASICs.

Given this research gap, we propose an FPGA-based hardware accelerator for performing LWE encryption and decryption using the TFHE scheme. The hardware can encrypt a plaintext (8-bit to 64-bit) into an LWE ciphertext and then decrypt an LWE ciphertext back into the plaintext. Input plaintext of size n -bit, where $n = 8, 16, 32, 64$, can be processed serially. The proposed method is useful for efficient FHE implementations for secure cloud, secure IoT communication, and secure edge computing platforms.

II. BACKGROUNDS

This section discusses the mathematical foundations of LWE encryption and decryption in the Torus Fully Homomorphic Encryption scheme. This section is based on Ilaria Chillotti's explanation about TFHE in [8] and [10].

A. LWE Encryption

Suppose we have a message in the form of plaintext m that we want to encrypt. m can be a single bit, a modular integer, or a real in the interval. In this implementation, we usually use m as an integer or fixed-point real. To encrypt the messages, we need two important parameters: The first one is a secret key with dimension n in the form of a vector:

$$\vec{s} = (s_0, s_1, s_2, \dots, s_{n-1}), \quad (1)$$

each element of secret key vector $\vec{s}$ is a binary number and $\vec{s} \in (0, 1)^n$ is either 0 or 1. Second, we need a vector of random elements from Z_q or integers modulo q as:

$$\vec{a} = (a_0, a_1, a_2, \dots, a_{n-1}), \quad (2)$$

For example, if we have $q = 11$, then all the integers modulo q are $Z_{11} = (0, 1, 2, 3, 4, 5, \dots, 10)$. This means elements of vector $\vec{a}$ are chosen between this range. Only after we have those two parameters, we can finally obtain the ciphertext c of plaintext m using this formula:

$$c = \text{Encrypt}_{\vec{s}}(\Delta m) = (\vec{a}, b) \quad (3)$$

ciphertext c is a set consisting of two elements: The first is vector $\vec{a}$, and the second is the scalar or integer b . b is a scalar resulted from multiplication of vector $\vec{a}$ and $\vec{s}$ calculated as:

$$b = \sum_{i=0}^{n-1} a_i \cdot s_i + \Delta m + e \quad (4)$$

Δm is the result of mapping a plaintext bit to the torus32 domain. We will discuss this further in Section III-C. An error term e must be added to obscure Δm . The error value e is chosen to follow a standard Gaussian distribution.

B. LWE Decryption

The decryption of LWE ciphertext c is carried out in two steps. Based on Equation (4), in the ciphertext, we already have information about vector $\vec{a}$ and scalar b . With this information, the first step is to calculate $\Delta m + e$ as follow:

$$\Delta m + e = b - \sum_{i=0}^{n-1} a_i \cdot s_i. \quad (5)$$

Second, we need to floor the value of $\Delta m + e$ to get the Δm , then divide it by Δ to get the plaintext bit m :

$$m = \frac{\lfloor \Delta m + e \rfloor}{\Delta}. \quad (6)$$

III. PROPOSED METHOD

A. Software-Hardware Interaction

The proposed design features a software-controlled hardware accelerator implemented on the PYNQ-Z1 FPGA. The accelerator is developed at the register-transfer level (RTL) and synthesized using Xilinx Vivado. The PYNQ-Z1 integrates an

ARM processor capable of running a Python kernel, which we utilize to perform input preprocessing and control the hardware execution via Overlay, a PYNQ hardware abstraction library. Algorithm 1 presents the software-hardware interaction flow, including data preparation, accelerator invocation, and result retrieval.

Algorithm 1: Software Flow to Control Hardware

Input: μ : encryption parameter,
 M_{size} : message size,
 $Plaintext$: 32-bit integer

Output: $Decrypted_Plaintext$: 32-bit integer

Encryption Phase: begin

```
// Load encryption bitstream as python object
overlay ← Overlay(encrypt.bit)
dma ← overlay.axi_dma for  $i \leftarrow 0$  to  $M_{size} - 1$ 
do
    enc_in[i] ← allocate buffer for encryption input
    enc_out[i] ← allocate buffer for encryption output
    // Feed plaintext and input params to buffer
    Initialize enc_in[i] using  $\mu$  and Plaintext
    // Transfer buffer to encryption hardware in
    dma[i].send(enc_in[i])
    // Transfer encryption hardware out to buffer
    dma[i].recv(enc_out[i])
// Delete python object for encryption
overlay.delete()
```

Decryption Phase: begin

```
// Load decryption bitstream as python object
overlay ← Overlay(decrypt.bit);
dma ← overlay.axi_dma;
for  $i \leftarrow 0$  to  $M_{size} - 1$  do
    dec_in[i] ← allocate buffer for decryption input
    dec_out[i] ← allocate buffer for decryption output
    // Feed ciphertext and params to buffer Copy
    enc_out[i] into dec_in[i] Append decryption
    parameters to dec_in[i]
    // Transfer buffer to decryption hardware in
    dma[i].send(dec_in[i])
    // Transfer decryption hardware out to buffer
    dma[i].recv(dec_out[i])
// Delete python object for decryption
overlay.delete()
```

return $Decrypted_Plaintext \leftarrow$ Extracted dec_out

B. Hardware Architecture

As shown in Algorithm 1, we implemented the encryption and decryption hardware separately. Figure 1 presents the hardware architecture of our LWE encryption and decryption system. The architecture is designed based on the mathematical foundations explained in Section II. The LWE encryption

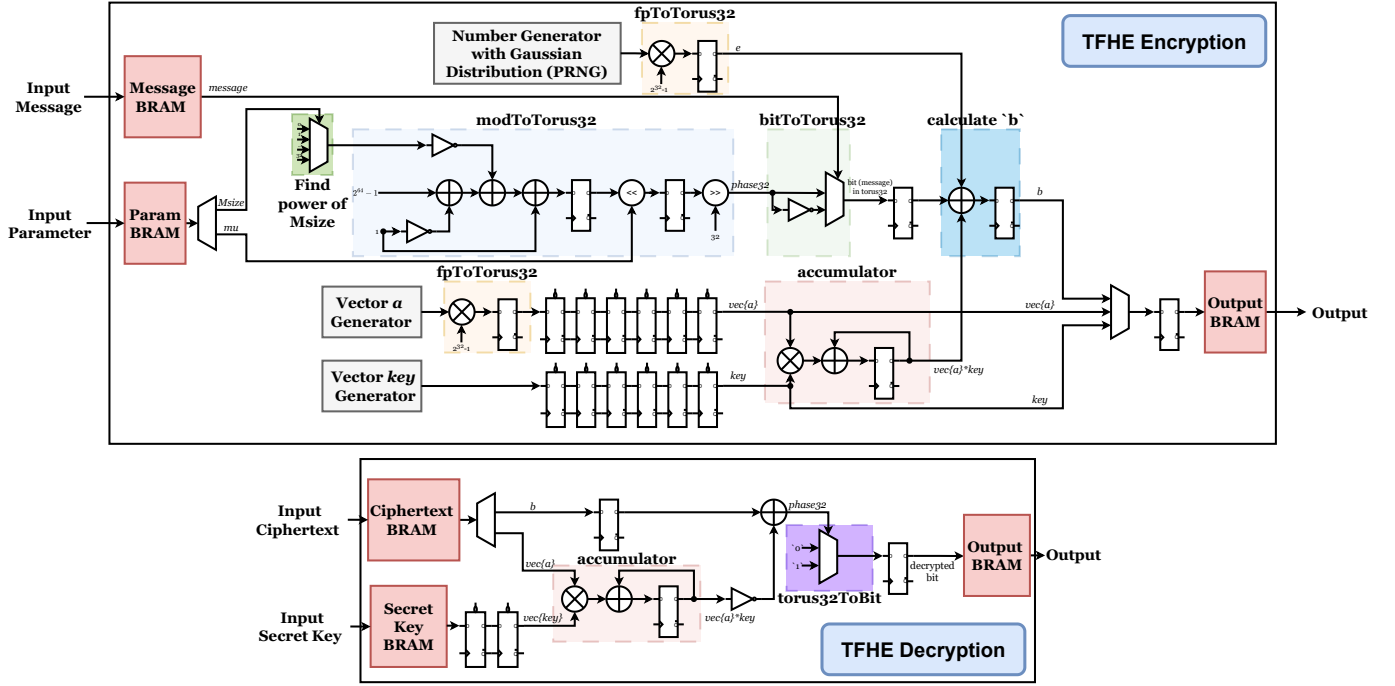


Fig. 1: Hardware architecture for encryption and decryption

hardware receives three inputs. The first input is the plaintext bit to be encrypted (m). The second input is μ , a unique character in the message space mapped to the torus domain. The third input is M_{size} , representing the number of discrete regions within the torus. We will discuss this further in Section III-C.

The Vector Key Generator module generates elements of $\vec{s}$ according to Equation (1). The Vector a Generator module generates elements of $\vec{a}$ according to Equation (2). The PRNG module produces the error value e used in Equation (4). The outputs from these three number generators are still in the real or integer domain and must be converted to the torus32 domain. The fpToTorus32 module performs this conversion for e and $\vec{a}$.

It is not necessary to convert the elements of $\vec{s}$ to the torus32 domain. This is because of how $\vec{s}$ is used in LWE encryption and decryption. $\vec{s}$ participates in dot product with $\vec{a}$, as shown in Equation (4) and Equation (5). As defined in Equation (1), elements of $\vec{s}$ consist only of binary values, either 0 or 1. Multiplying any number by 0 results in 0. Multiplying any number by 1 leaves the number unchanged. Therefore, as long as $\vec{a}$ is represented in the torus32 domain, the resulting dot product $\vec{a} \cdot \vec{s}$ will also be in the torus32 domain. Therefore, we don't need to convert $\vec{s}$ to the torus32 domain.

Since LWE operates in the boolean scale, the bitToTorus32 module is used to map the plaintext bit to a suitable Δm value in the torus32 domain. The Δm is computed using the modToTorus32 module, which maps unique characters

in the plaintext message space to torus32, commonly referred to as Δm . The modules fpToTorus32, bitToTorus32, and modToTorus32 are essential to ensure all operations are carried out within the torus32 domain. These modules will be discussed in more detail in Section III-C.

The accumulator module performs the multiplication of $\vec{a}$ and $\vec{s}$ and accumulates the results according to Equation (4). The final scalar b following Equation (4) is obtained by summing the outputs of fpToTorus32, bitToTorus32, and accumulator. The output of the LWE encryption hardware is a data stream consisting of b , $\vec{a}$, and $\vec{s}$, which is stored in BRAM.

Following Algorithm 1, after encryption is completed, the decryption hardware is invoked in software. The decryption hardware receives two inputs from the encryption output: the LWE ciphertext ($\vec{a}, b$) and the secret key $\vec{s}$. The accumulator module performs the multiplication of $\vec{a}$ and $\vec{s}$ and accumulates the results according to Equation 5. This accumulated value is then subtracted from b , yielding the $\Delta m + e$, following Equation 5. Finally, the torus32ToBit module maps $\Delta m + e$ to a binary value of 0 or 1, successfully decrypting the plaintext bit. The torus32ToBit is a simplification of Equation (6), applicable to LWE encryption where plaintext consists of only binary digits (0 and 1).

C. How Torus32 Works in Our Proposed Design

Before discussing this further, we present the definitions of several important notations, such as $\mathbb{Z}$, $\mathbb{R}$, $\mathbb{T}$, and $\mathbb{T}_N[X]$ in

Table I. fpToTorus32 converts a 32-bit fixed-point value from the real number domain to the torus32 domain. The number 32 in torus32 refers to the bit-width used for the representation.

Example 3.1. Suppose we have a 32-bit fixed-point number in Q15.16 format $x = 2.25, x \in \mathbb{R}$ and its binary representation is: $x = 0000000000000010.0100000000000000, x \in \mathbb{R}$. Thus, the corresponding value of x in the torus32 domain is $x_{\text{torus32}} = 0.25, x \in \text{torus32}$. In binary form, this becomes $x_{\text{torus32}} = 0000000000000000.0100000000000000, x \in \text{torus32}$. ■

Thus, the torus32 representation of the real number 1.25 is the same as that of 2.25, 3.25, or even 100.25, as long as the fractional part remains unchanged. Since the maximum fractional value is typically within the range $[0, 0.99999]$, the fixed-point representation in the torus32 domain is inherently bounded to the interval $[0, 1)$. Therefore, torus32 can be visualized as a ring, as illustrated in Figure 2.

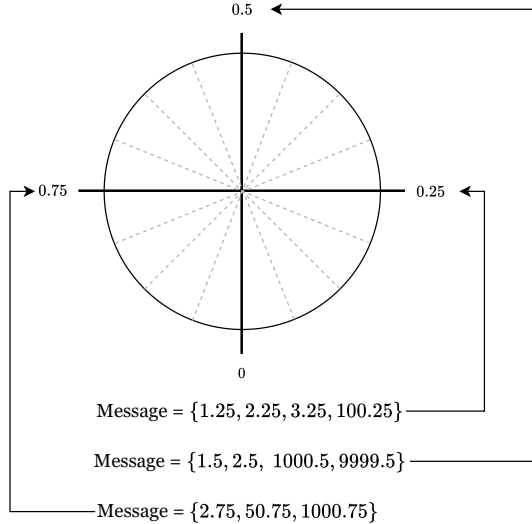


Fig. 2: A ring to map fixed-point values to the torus32 domain

modToTorus32 module is used to find the message representation in the torus32 domain. Since LWE operates on a boolean scale, bitToTorus32 is used to convert messages from the integer domain to the torus32 domain based on the output of modToTorus32. Hence, modToTorus32 plays an important role in determining the message value within the torus32 domain.

Consider the architecture shown in Figure 1. modToTorus32 has two types of input, namely μ and M_{size} . μ is an integer that represents the maximum value among the unique characters that make up the plaintext. Plaintext consists of integers or real numbers, so the unique characters forming the message are integers. For example, if the plaintext is 5573, then it is composed of three integers: 5, 7, and 3, making $\mu = 7$. If the

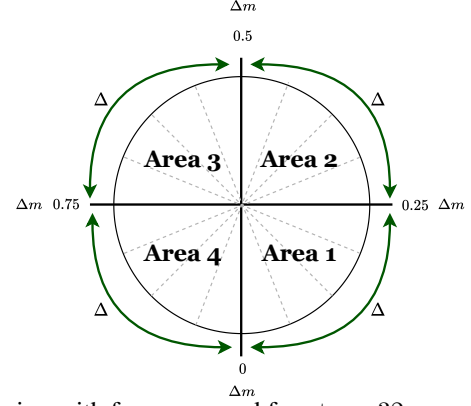


Fig. 3: A ring with four areas and four torus32 representations

plaintext is 100000, then the message consists of 0 and 1, thus $\mu = 1$. Since LWE operates on a Boolean scale, the message typically consists only of 0 and 1, hence $\mu = 1$.

M_{size} denotes the number of discrete areas within torus32. As an example, refer to the illustration of torus32 in Figure 2. In this figure, torus32 is divided into 4 areas, as shown in Figure 3. Each area has an interval of size Δ . The values that delimit each area represent torus32 values mapped from real or integer domains, which we call torus32 representations. Not all torus32 representations are results of plaintext mappings. The torus32 representations that corresponds to the plaintext mappings is called Δm .

The small values within each area represent $\Delta m + e$, where e is an error term. The error shifts Δm by some interval. If the error magnitude is smaller than Δ , then the floor of $\Delta m + e$ equals Δm . If the error is greater than Δ , then the floor of $\Delta m + e$ does not equal Δm , but rather corresponds to a different torus32 representations. That is :

$$\lfloor \Delta m + e \rfloor = \begin{cases} \Delta m, & \text{if } e < \Delta \\ \neq \Delta m, & \text{if } e > \Delta \end{cases}$$

If $e > \Delta$, the plaintext obtains a torus32 representations different from Δm , resulting in a decryption error.

The value of M_{size} can be arbitrary. For LWE, we only need to determine the torus32 representations of 0 and 1, meaning the minimum M_{size} required is 2. However, there are problems if we choose an M_{size} that is either too small or too large.

With a small M_{size} of 2, the only possible torus32 representations are 0 and 0.5. Consequently, there are only two possible mappings of $p = \{0, 1\}$ into the torus32 domain: either $\{0, 1\} \rightarrow \{0, 0.5\}$ or $\{0, 1\} \rightarrow \{0.5, 0\}$. Simply trying both possibilities makes it easy to deduce the torus32 representations of the plaintext (Δm).

In contrast, with a sufficiently large M_{size} , such as 16, there are sixteen possible torus32 representations. This results in many potential mappings of $p = \{0, 1\}$ into the torus32

TABLE I: Notations related to Torus

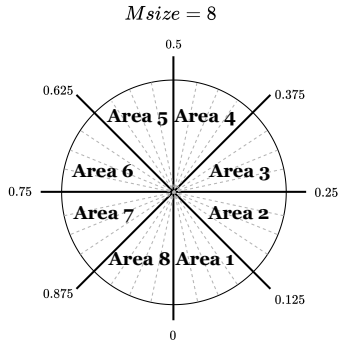
Notation	Description	Example
$\mathbb{Z}$	Set of Integers	$\mathbb{Z} = (1, 2, 3, \dots)$
$\mathbb{R}$	Set of Real Numbers	$\mathbb{R} = (0.25, 0.50, 1.25, 1.5, 2.25, 2.5, \dots)$
$\mathbb{T}$	Set of Torus $\mathbb{R}/\mathbb{Z}$, which is set of Real Numbers $\mathbb{R} \bmod 1$	$\mathbb{T} = (0.25, 0.5, 0.75, 0.999, \dots)$
$\mathbb{T}_N[X]$	A torus polynomial of degree $(N - 1)$ as $\mathbb{R}[X]/(X^N + 1) \bmod 1$ whose coefficient come from $\mathbb{T}$	$\mathbb{T}_N[X] = 0.3 + 0.5X + 0.75X^2 + \dots + 0.2X^{N-1}$

TABLE II: Speed Comparison for Encryption and Decryption (in ms)

Plaintext Size	Encryption			Decryption		
	Software [9] (ms)	Proposed (ms)	Speedup	Software [9] (ms)	Proposed (ms)	Speedup
8-bit	15.541	0.368	42×	4.315	0.427	10×
16-bit	16.149	0.736	22×	4.315	0.854	5×
32-bit	14.914	1.104	13.5×	5.336	1.281	4.165×
64-bit	13.371	1.472	9.1×	5.741	1.708	3.36×

TABLE III: FPGA Resource Utilizations

Resource	Encrypt (%)	Decrypt (%)
LUT	59.11	52.22
LUTRAM	19.60	18.34
FF	37.61	34.90
BRAM	60.00	48.57
BUFG	6.25	6.25

Fig. 4: Torus32 with $M_{\text{size}} = 8$ used in our proposed design

domain. For example, $\{0, 1\} \rightarrow \{0, 0.9375\}$ or $\{0, 1\} \rightarrow \{0.25, 0.75\}$, among many others. The remaining fourteen torus32 representations serve as decoys, making it more difficult to guess the torus32 representations (Δm).

Additionally, a small M_{size} results in the torus being divided into only a few regions, allowing the error to grow significantly. On the other hand, a large M_{size} produces many regions, each with a small width, where even a slight amount of error can cause the torus32 representations of the plaintext to shift.

Therefore, to strike a balance such that the number of discrete regions in torus32 is neither too many nor too few, in our proposed design, we choose $M_{\text{size}} = 8$. Figure 4 provides an illustration of the torus32 domain used in this paper.

To determine the torus32 representations of 0 and 1 in the

torus32 domain shown in Figure 4, Equation (1) is used to compute their corresponding (Δm) values:

$$\Delta m = \text{plaintext}_{\text{bit}} \times \frac{2^{(32)}}{M_{\text{size}} \times 2^{32}} \quad (1)$$

Example 3.2 illustrates the calculation of Δm for 0 and 1 when $M_{\text{size}} = 8$

Example 3.2. Suppose that we have a 32-bit fixed-point number in $Q15.16$ format $x = 2.25$, $x \in \mathbb{R}$ with its binary representation: $x = 0000000000000010.0100000000000000$, $x \in \mathbb{R}$. We want to encrypt x using TFHE LWE encryption. Hence, we need to find the torus32 representations or the Δm values of 0 and 1. Suppose $M_{\text{size}} = 8$ then : $\Delta m_{m=0} = 0 \times \frac{2^{(32)}}{M_{\text{size}} \times 2^{32}} \approx 0$ and $\Delta m_{m=1} = 1 \times \frac{2^{(32)}}{M_{\text{size}} \times 2^{32}} \approx 0.125$. ■

IV. EXPERIMENTAL RESULTS

A. Performance

The hardware design is implemented using the Verilog HDL language and synthesized and deployed on the PYNQ-Z1 FPGA platform using Xilinx Vivado. The hardware operates at a clock frequency of 100 MHz.

The software described in [9] is an implementation of the mathematical foundations of TFHE as presented in [5]. In our experiments, we evaluate the performance of this software running on an AMD Ryzen 7 CPU with 16 GB of memory. The TFHE software is compiled using the tfhe-splios-fma backend, which employs SIMD optimizations such as FMA and AVX2 instructions to accelerate homomorphic operations. However, this backend executes single-threaded, utilizing only one CPU core during runtime. CPU profiling confirms that only a single thread is active throughout execution. The base clock frequency of the AMD Ryzen 7 CPU used in this evaluation is 2 GHz, which is 20 times higher than the 100 MHz clock frequency of the proposed hardware. Therefore, the software execution times reported in Table II reflect single-threaded

TABLE IV: Comparisons with Recent State-Of-The-Art TFHE Methods

Ref.	Algorithms	Components	Implementation	P	Q	Latency	Max Freq.	Resource Consumption		
								LUT	FF	DSP
Ref [7]	Bootstrapping	CRT, ICRT, DR, MNTT, NTT/INTT	ALT (ASIC)	N/A	N/A	3.76 ms	N/A	623K	256K	2144
Ref [8]	Bootstrapping	NTT, Montgomery Modular Multiplication	ZYNQ UltraScale +MPSoC ZCU102	N/A	N/A	1.9 ms	300 MHz	216K	238K	1024
Ref [6]	Bootstrapping	NTT/INTT	UltraScale+ VCU128	N/A	N/A	6.62 ms	200 MHz	544K	843K	2881
Ref [11]	Bootstrapping	NTT/INTT	UltraScale+ VCU128	N/A	N/A	249 ms	N/A	N/A	N/A	N/A
Ours	Encryption	Modular Reduction, Bit and Fixed-Point Converter to Torus32, Accumulator	Xilinx PYNQ-Z1	2	2 ³²	1.472 ms	100 MHz	31.45k	40.02k	0
Ours	Decryption	Accumulator and Torus23	Xilinx PYNQ-Z1	2	2 ³²	1.708 ms	100 MHz	27.78K	37.14k	0

performance at a significantly higher clock rate than the FPGA-based implementation.

Table II presents a comparison of encryption and decryption execution times between the single-threaded software from [9] and the proposed hardware design. The reference software, developed in C++ and C by Ilaria Chillotti and colleagues, was evaluated using equal vector sizes for $\vec{a}$ and $\vec{s}$, denoted by n .

Table III shows the consumption of FPGA resources by hardware. It can be seen that the proposed hardware does not consume many FPGA resources. Since resource usage is low, power consumption will also be low. This is clearly advantageous because it means that the hardware is lightweight and can operate with low power consumption.

B. Comparisons with Previous State-Of-The-Art Methods

Table IV presents a comparison between the proposed hardware and previous research on TFHE. Most prior studies have focused on bootstrapping, as seen in the works [6], [7], [8], and [11], without dedicated hardware implementations for encryption and decryption in the TFHE scheme. This indicates that to date, research on hardware acceleration for LWE encryption and decryption in TFHE remains very limited, and the hardware proposed in this study is among the first.

In terms of latency, the bootstrapping methods in previous studies show significant performance variations. The work [11] reports the slowest execution time of 249 ms, while the work [6] has a much faster execution time of 6.62 ms. Other implementations, such as the work [8], record an execution time of 1.9 ms, which is even faster. In comparison, our proposed hardware in this study achieves an execution time of 1.472 ms for encryption and 1.708 ms for decryption, making it competitive with existing bootstrapping implementations despite targeting a different cryptographic process.

V. CONCLUSION

In this work, we proposed a novel TFHE-based encryption and decryption hardware accelerators implemented in PYNQ-Z1 Xilinx FPGA. The hardware accelerator that has been developed already provides faster encryption and decryption

functions compared to the referenced software up to 40× and 10× speed-up for the encryption and the decryption, respectively. The designed hardware accelerator operates at a clock frequency of 100 MHz and utilizes FPGA resources efficiently in both area and power. The hardware accelerator for LWE encryption and decryption using the TFHE scheme has been successfully developed. In the future, the ring-LWE-based (RLWE) and the ring-Gentry-Sahai-Waters-based (RGSW) TFHEs are very interesting to be implemented in FPGAs or ASICs.

ACKNOWLEDGEMENT

This work is supported by ITB Research Program: RK3P Stage II 2025.

REFERENCES

- [1] R. L. Rivest, L. Adleman, M. L. Dertouzos *et al.*, “On data banks and privacy homomorphisms,” *Foundations of secure computation*, vol. 4, no. 11, pp. 169–180, 1978.
- [2] R. L. Rivest, A. Shamir, and L. Adleman, “A method for obtaining digital signatures and public-key cryptosystems,” *Communications of the ACM*, vol. 21, no. 2, pp. 120–126, 1978.
- [3] C. Gentry, *A fully homomorphic encryption scheme*. Stanford university, 2009.
- [4] L. Ducas and D. Micciancio, “FHEw: bootstrapping homomorphic encryption in less than a second,” in *Annual international conference on the theory and applications of cryptographic techniques*. Springer, 2015, pp. 617–640.
- [5] I. Chillotti, N. Gama, M. Georgieva, and M. Izabachène, “Tfhe: fast fully homomorphic encryption over the torus,” *Journal of Cryptology*, vol. 33, no. 1, pp. 34–91, 2020.
- [6] T. Kong and S. Li, “Hardware acceleration and implementation of fully homomorphic encryption over the torus,” *IEEE Transactions on Circuits and Systems I: Regular Papers*, 2023.
- [7] X. Hu, Z. Li, Z. Wang, and X. Lu, “Alt: Area-efficient and low-latency fpga design for torus fully homomorphic encryption,” *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, 2024.
- [8] J. Zhang, A. Cui, and Y. Jin, “Acceleration of the bootstrapping in tfhe by fpga,” *IEEE Transactions on Emerging Topics in Computing*, 2024.
- [9] I. Chillotti, “TFHE Fast Fully Homomorphic Encryption over the Torus — tfhe.github.io,” <https://tfhe.github.io/tfhe/>, [Accessed 02-11-2024].
- [10] —, “Tfhe deep dive,” <https://youtu.be/LZuEr4jpyUw?si=Ik4D5DQbyVoAb588>, 2022, [Accessed 02-11-2024].
- [11] J. Takeshita, D. Reis, T. Gong, M. Niemier, X. S. Hu, and T. Jung, “Accelerating finite-field and torus fhe via compute-enabled (s) ram,” *IEEE Transactions on Computers*, 2023.